DOCUMENT 4 / 4 — COMPLÉMENTS

Schémas complémentaires

Trois zooms techniques qui approfondissent le document « Architecture technique » : la stratégie d'agrégation des sources de prix, l'agent IA interne (ChatBot boutique) et la carte des dépendances entre modules.

Agrégateurs de données | Agent IA — function calling | Dépendances entre modules

Client
Gérant boutique Pokémon (30 m²)

Complète
Doc 2 « Architecture technique » §4-5

Source
Coffre Obsidian — 04-Data-Sources, 03-Metier

Généré le
2 septembre 2026

SOMMAIRE

Contenu du document

01 **Agrégateurs de données** — Qui fournit quoi, comment, et ce qui est encore planifié

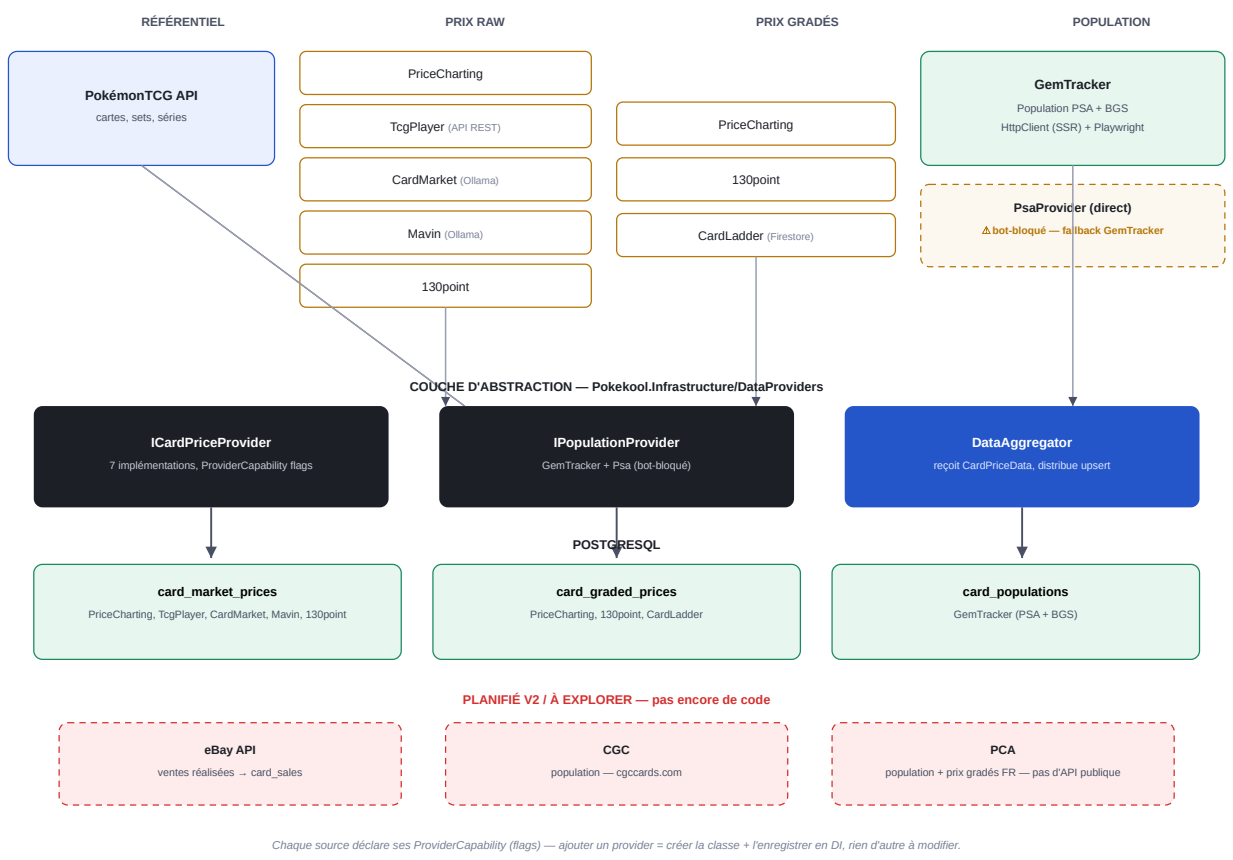
02 **Agent IA interne — ChatBot boutique** — Architecture function calling, outils, confirmation humaine

03 **Modules & leurs liens** — Ce qui dépend de quoi, concrètement, dans le code

01 — DONNÉES

Agrégateurs de données

Il n'existe pas une API unique exposant cartes, grades et prix en temps réel : chaque fournisseur ne couvre qu'un rôle précis. Le backend agrège **10 sources actives** derrière deux interfaces (`ICardPriceProvider`, `IPopulationProvider`) — ajouter une source se limite à créer une classe et l'enregistrer en DI.



10 sources actives agrégées derrière deux interfaces provider — 3 sources planifiées V2 non codées à ce jour.

Endpoints de synchronisation

Endpoint	Commande	Met à jour
`POST /admin/sync/catalog`	`SyncPokemonTcgCommand`	`pokemon_sets`, `pokemon_cards`
`POST /admin/sync/prices`	`SyncCardPricesCommand`	`card_market_prices`, `card_graded_prices`
`POST /admin/sync/population`	`SyncCardPopulationCommand`	`card_populations`

Permission `cards:sync` requise. Body `{ "cardIds": [...] }` — `null` = toutes les cartes. Indicateur dérivé : `GemRate10 = Pop10 / TotalGraded × 100` (rareté du grade parfait, pilote la prime de prix).

MISE À JOUR DEPUIS LE DOCUMENT « ARCHITECTURE TECHNIQUE »

Cette carte, basée sur `04-Data-Sources/Strategie-Agregation.md`, est plus précise que le §4 du document 2 : elle isole **GemTracker** (source réellement utilisée pour la population PSA/BGS) du **PsaProvider direct** (bloqué par anti-bot), et distingue clairement ce qui est *planifié* (eBay, CGC, PCA) de ce qui est *en production*.

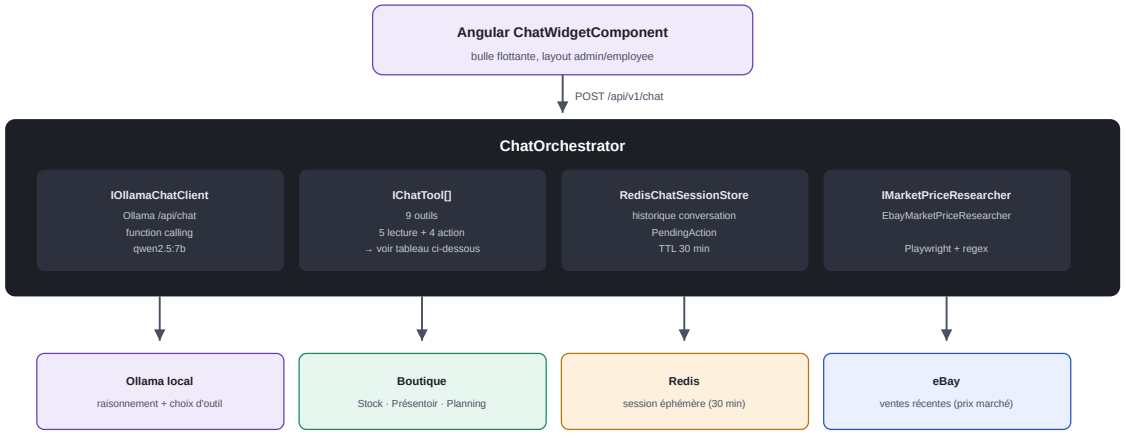
02 — AGENT IA

Agent IA interne — ChatBot boutique

Assistant conversationnel réservé à l'admin et aux employés (bulle 🤖 flottante) pour piloter stock, présentoir et planning en langage naturel, via **function calling** sur un modèle local (Ollama, `qwen2.5:7b`). Aucune action n'est exécutée sans confirmation explicite.

STATUT RÉEL — SPÉCIFIÉ, PAS ENCORE CODÉ

`03-Metier/Assistant-IA-Feature.md` décrit ce chatbot au présent (« l'agent peut… ») mais la table de priorités de `Automatisations-LLM.md` le classe 🔲 **À faire** (contrairement au scan collection, ✅ fait — voir doc « Architecture technique » §5). Ce schéma documente la **cible de conception**, pas un système en production — à corriger dans le vault pour éviter toute ambiguïté à la reprise.



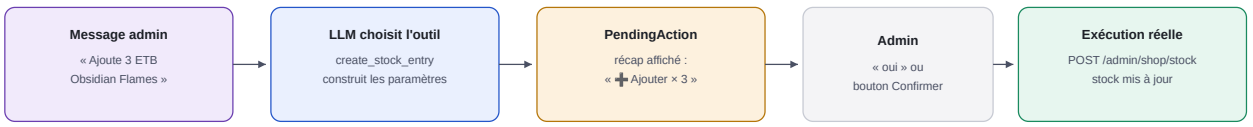
Architecture cible du ChatOrchestrator — un hub, quatre dépendances, un seul point d'entrée HTTP.

Les 9 outils (function calling)

Outil	Type	Description
`search_shop_stock`	Lecture	Cherche dans le stock par nom de carte/produit
`search_shop_products`	Lecture	Cherche sur le présentoir
`get_planning`	Lecture	RDV sur une période, filtrable par statut
`get_market_price`	Lecture	Prix eBay ventes récentes d'une carte
`open_camera`	Lecture	Redirige vers le scanner de collection
`create_stock_entry`	Action	Ajoute un article au stock — **confirmation requise**

Outil	Type	Description
`create_shop_product`	Action	Met un article en vente — **confirmation requise**
`update_appointment_status`	Action	Change le statut d'un RDV — **confirmation requise**
`reschedule_appointment`	Action	Déplace un RDV — **confirmation requise**

Human-in-the-loop — aucune action sans « oui »



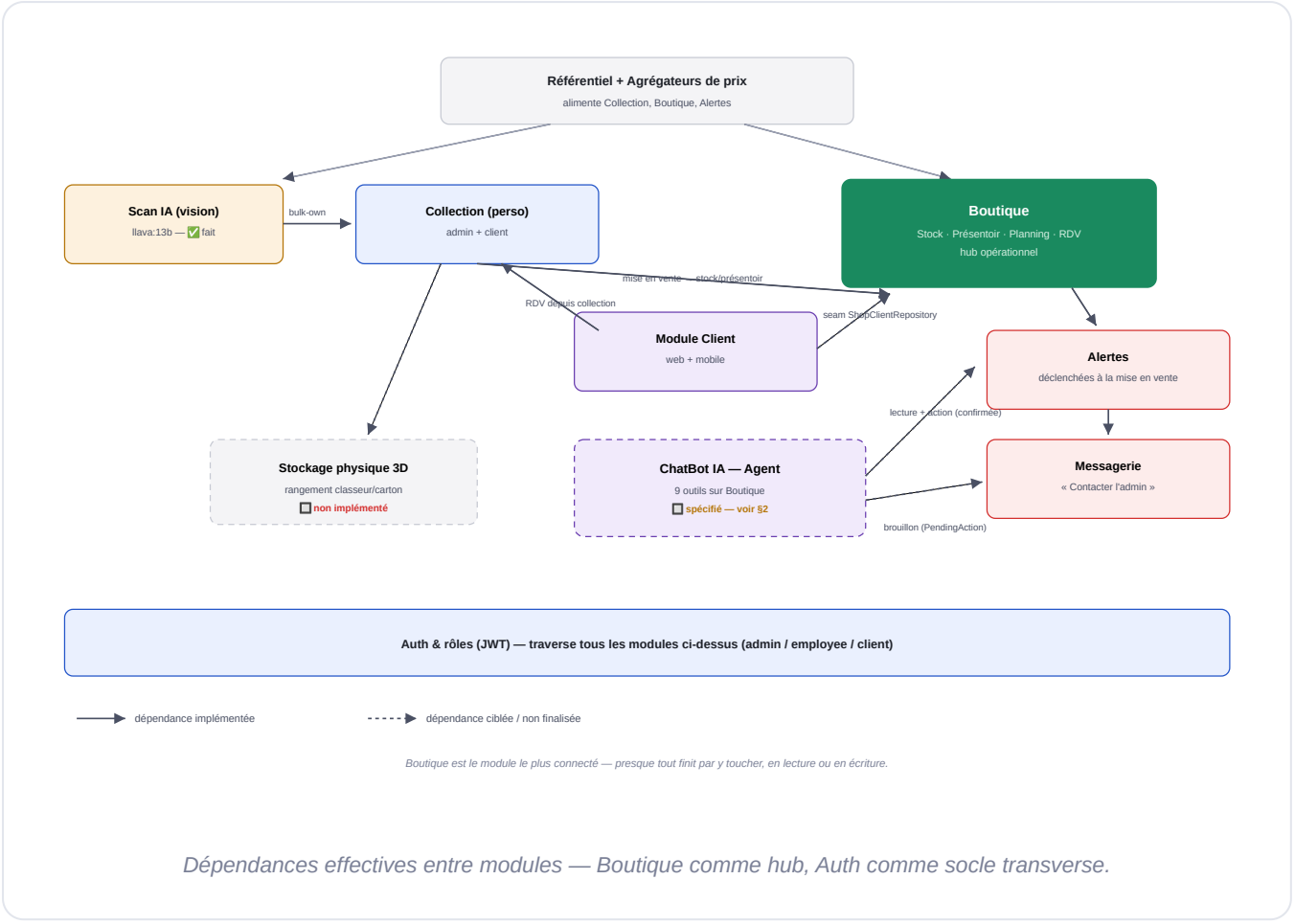
Un « non », ou un silence de 30 minutes, annule la PendingAction — rien n'est exécuté par défaut.

Ambiguïté (ex. « 35 deoxys ») → l'outil de recherche liste les variantes, le LLM demande de choisir avant de continuer.

03 — DÉPENDANCES

Modules & leurs liens

Vue différente du document « Vue d'ensemble » §3 : ici, ce ne sont plus les interfaces utilisateur qui structurent le schéma, mais les **dépendances réelles** entre modules côté code — qui appelle qui, qui déclenche quoi.



Dépendances effectives entre modules — Boutique comme hub, Auth comme socle transverse.

Détail des liens

De	Vers	Déclencheur	Statut
Scan IA	Collection	`POST /collection/bulk-own` après confirmation	Fait
Collection	Boutique	Admin marque une carte « à vendre » → devient stock/présentoir	Fait
Collection	Module Client	Prise de RDV depuis une carte de la collection client	Fait
Module Client	Boutique	Dépendance isolée dans `ShopClientRepository` (seam)	Fait
Boutique	Alertes	Mise en vente → recherche des alertes actives sur la carte	À faire

De	Vers	Déclencheur	Statut
Boutique	Messagerie	Bouton « Contacter l'admin » depuis le catalogue	À faire
ChatBot IA	Boutique	9 outils — 5 lecture, 4 action confirmée	Spécifié
ChatBot IA / AI-Brain	Messagerie	Brouillon de réponse, même pattern `PendingAction`	Spécifié
Collection	Stockage physique	Rangement classeur/carton d'un exemplaire	À faire
Auth	Tous les modules	JWT + rôle (admin/employee/client) sur chaque endpoint	Fait

Sources — coffre Obsidian « Projet_Pokemon » : 04-Data-Sources/Strategie-Agregation.md, 03-Metier/Assistant-IA-Feature.md, 03-Metier/Automatisations-LLM.md, 03-Metier/Alertes-Feature.md, 03-Metier/Messagerie-Feature.md, 03-Metier/Collection-Feature.md, 00-Index/HOME.md. Document généré le 2 septembre 2026.